

AI-Powered HR Assistant — Complete System Workflow Description

1 System Overview

The system is an AI-powered HR assistant designed to automate common HR tasks such as answering employee questions about company policies and personal HR data (e.g., payroll, leave balance, working hours).

The architecture follows a **controlled agent-based design**, where:

- The **LLM is used only for reasoning and response generation**
- All **data access is handled through secure backend tools/APIs**
- **Authorization and security are enforced outside the LLM**

The system integrates:

- A **structured database** (employee-specific data)
- A **vector database** (company documents and policies)
- A **memory layer** (for conversation context)

2 Core Components

2.1 User Interface

- Chat interface (web or mobile)
- Employees submit natural language queries

2.2 API Gateway

Responsibilities:

- Authentication validation (SSO/OAuth)
- Rate limiting
- Request logging

2.3 Authorization Layer

Implements:

- RBAC (Role-Based Access Control) or ABAC (Attribute-Based Access Control)

Outputs:

- Allowed tools for the user
- Data access scope (e.g., `user_id` restrictions)

Important:

- This layer enforces security BEFORE any LLM interaction

2.4 Orchestrator (Agent Controller)

Acts as the central controller of the system.

Responsibilities:

- Inject user context and permissions into the workflow
- Control which tools are available to the LLM
- Manage multi-step execution for complex queries
- Coordinate memory usage

2.5 Memory Manager

Handles conversation context.

Types:

- Short-term memory: recent messages (last N interactions)
- Long-term memory: summarized past conversations

Responsibilities:

- Retrieve relevant history before LLM call
- Store new interactions after response generation

Storage options:

- Relational DB (PostgreSQL)
- NoSQL DB (MongoDB)
- Vector DB (for semantic retrieval)

Constraints:

- Per-user isolation
- Encryption of stored conversations
- Avoid storing sensitive raw data unnecessarily

2.6 LLM (Reasoning Engine)

Role:

- Understand user intent
- Decide which tools to use
- Generate natural language responses

Inputs:

- User query
- Allowed tools
- System rules (strict instructions)
- Retrieved memory context

Important constraints:

- The LLM does NOT access databases directly
- The LLM can only call predefined tools

2.7 Tool Layer (Secure APIs)

All data access is performed through controlled tools.

Types of tools:

1. Structured Data Tools (SQL-backed):

- `get_my_salary(user_id)`
- `get_leave_balance(user_id)`
- `get_work_hours(user_id)`

2. Document Retrieval Tool (RAG):

- `search_hr_policy(query)`

Tool properties:

- Enforce authorization internally
- Never expose raw database access
- Return only filtered, minimal data

2.8 Data Sources

1. Structured Database:
 - Stores employee-specific data
 - Example: PostgreSQL
2. Vector Database:
 - Stores embedded company documents
 - Used for semantic search (RAG)

2.9 Logging and Monitoring

Tracks:

- User queries
- Tools used
- Data accessed
- System responses

Used for:

- Auditing
- Debugging
- Compliance

3 End-to-End Workflow

Step 1: User Request

The user submits a query through the interface.

Step 2: Authentication & API Gateway

- Validate user identity
- Apply rate limiting
- Log request

Step 3: Authorization

- Determine what the user is allowed to access
- Define:
 - Allowed tools
 - Data scope (e.g., only their own data)

Step 4: Memory Retrieval

- Fetch relevant conversation history
- Provide context for follow-up queries

Step 5: Orchestrator Setup

- Prepare LLM input:
 - User query
 - Memory context
 - Allowed tools
 - System rules

Step 6: LLM Reasoning & Planning

- Interpret the query
- Generate a plan of tool calls

Example:

```
[  
get_my_leave_balance,  
search_hr_policy("vacation policy")  
]
```

Step 7: Tool Execution

- Backend performs secure queries
- Authorization is enforced again
- Only relevant data is returned

Step 8: Multi-Source Aggregation

- Combine outputs from different tools
- Prepare structured data for response generation

Step 9: Final Response Generation

- LLM generates a natural language response using:
 - Tool outputs
 - Memory context

Step 10: Post-Processing

- Validate response
- Remove sensitive or unintended information
- Format output

Step 11: Memory Update

- Store the interaction (query + response)
- Optionally summarize for long-term memory

Step 12: Logging & Audit

- Record full trace:
 - Query
 - Tools used
 - Data accessed

4 Handling Complex Queries

For queries requiring multiple data sources:

Example: “Compare my salary with the average salary in my department”

Execution:

1. `get_my_salary(user_id)`
2. `get_department_avg_salary(department)`
3. Combine results
4. Generate response

If unauthorized:

- The tool denies access
- The system returns a safe message

5 Security Principles

- LLM never directly accesses databases
- All data access is mediated by secure tools
- Authorization is enforced:
 - Before LLM execution

- Inside each tool
- Strict per-user data isolation
- Full audit logging
- Protection against prompt injection

6 Key Design Principles

- Separation of concerns:
 - LLM = reasoning
 - Backend = control
 - Tools = execution
- Deterministic and controlled data access
- Scalable and modular architecture
- Memory managed externally (not inside LLM)

7 Conceptual Model

- LLM = Assistant
- Backend = Controller
- Tools = Executors
- Memory = Context Manager
- Database = Secure Storage

8 Summary

This system ensures:

- Secure access to sensitive HR data
- Accurate responses using multiple data sources
- Context-aware conversations through memory
- Full control over LLM behavior via orchestration and tooling

The architecture is designed to be production-ready, scalable, and compliant with enterprise security standards.